Bloom Filter-Based MPSI

Weekly Progress Meeting 5 March 2026

Andra Alăzăroaie

Supervisor: Lilika Markatou

Daily Supervisor: Tjitske Koster

05-03-2026

Bibliography

[1] Jelle Vos, Jorrit van Assen, Tjitske Koster, Evangelia Anna Markatou, and Zekeriya Erkin. On the insecurity of bloom filter-based private set intersections. Cryptology ePrint Archive, 2024.

[3] Ou Ruan, Changwang Yan, Jing Zhou, and Chaohao Ai. A practical multiparty private set intersection protocol based on bloom filters for unbalanced scenarios. Applied Sciences, 13(24):13215, 2023.

[6] Vladimir Kolesnikov, Naor Matania, Benny Pinkas, Mike Rosulek, and Ni Trieu. Practical multi-party private set intersection from symmetric-key techniques.In Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, pages 1257–1272, 2017.

[7] Alireza Kavousi, Javad Mohajeri, and Mahmoud Salmasizadeh. Efficient scalable multi-party private set intersection using oblivious prf. In International Workshop on Security and Trust Management, pages 81–99. Springer, 2021.

[8] Florian Kerschbaum. Outsourced private set intersection using homomorphic encryption. In Proceedings of the 7th ACM Symposium on Information, Computer and Communications Security, pages 85–86, 2012.

[9] Casacuberta, Sílvia, Julia Hesse, and Anja Lehmann. "SoK: oblivious pseudorandom functions." In 2022 IEEE 7th European Symposium on Security and Privacy (EuroS&P), pp. 625-646. IEEE, 2022.

[10] Dong, Changyu, Liqun Chen, and Zikai Wen. "When private set intersection meets big data: an efficient and scalable protocol." In *Proceedings of the 2013 ACM SIGSAC conference on Computer & communications security*, pp. 789-800. 2013.

[11] Falzon, Francesca, and Evangelia Anna Markatou. "Re-visiting authorized private set intersection: A new privacy-preserving variant and two protocols." *Proceedings on Privacy Enhancing Technologies* (2025).

Progress

From the plan last week:

- Finalize and benchmark GBF
- Do some writing on the mitigations so far
- Continue with the Authorized MPSI mitigation (read some references of [11])

Progress

GBFs [10] adaptation to **Ruan et al. [6]** :

- Instead of the XOR of shares giving x : work in the same (multiplicative) group used by ElGamal, the shares are group elements
- Made a set with the indices generated by the hash functions to make sure that they are unique

Algorithm 1: $BuildGBF(S, n, m, k, H, \lambda)$

```
input  : A set S, n, m, k, λ, H = {h_0, ...h_{k-1}}
output: An (m, n, k, H, λ)-garbled Bloom filter GBF_S
GBF_S= new m-element array of bit strings;
for i= 0 to m − 1 do
    GBF_S[i]=NULL;                          // NULL is the special symbol that means ''no value''
end
for each x ∈ S do
    emptySlot = −1, finalShare= x;
    for i=0 to k-1 do
        j = h_i(x);                         // get an index by hashing the element
        if GBF_S[j]==NULL then
            if emptySlot ==−1 then
                emptySlot=j;                // reserve this location for finalShare
            else
                GBF_S[j] ←r {0,1}^λ;        // generate a new share
                finalShare=finalShare⊕GBF_S[j];
            end
        else
            finalShare=finalShare⊕GBF_S[j]; // reuse a share
        end
    end
    GBF_S[emptySlot]=finalShare;            // store the last share
end
for i= 0 to m − 1 do
    if GBF_S[i]==NULL then
        GBF_S[i] ←r {0,1}^λ;
    end
end
```

Progress

GBFs [10] adaptation to **Ruan et al. [6]** :

- Original Protocol:
  - The server multiplies the encrypted bins (encrypted 1's in the original protocol) with a blinded version of its element
  - Adds to the intersection if the decryption is the blinded element

Algorithm 1: $BuildGBF(S, n, m, k, H, \lambda)$

```
input  : A set S, n, m, k, λ, H = {h_0, ...h_{k-1}}
output: An (m, n, k, H, λ)-garbled Bloom filter GBF_S
GBF_S= new m-element array of bit strings;
for i= 0 to m − 1 do
    GBF_S[i]=NULL;                          // NULL is the special symbol that means ''no value''
end
for each x ∈ S do
    emptySlot = −1, finalShare= x;
    for i=0 to k-1 do
        j = h_i(x);                         // get an index by hashing the element
        if GBF_S[j]==NULL then
            if emptySlot ==−1 then
                emptySlot=j;                // reserve this location for finalShare
            else
                GBF_S[j] ←r {0,1}^λ;        // generate a new share
                finalShare=finalShare⊕GBF_S[j];
            end
        else
            finalShare=finalShare⊕GBF_S[j]; // reuse a share
        end
    end
    GBF_S[emptySlot]=finalShare;            // store the last share
end
for i= 0 to m − 1 do
    if GBF_S[i]==NULL then
        GBF_S[i] ←r {0,1}^λ;
    end
end
```

Progress

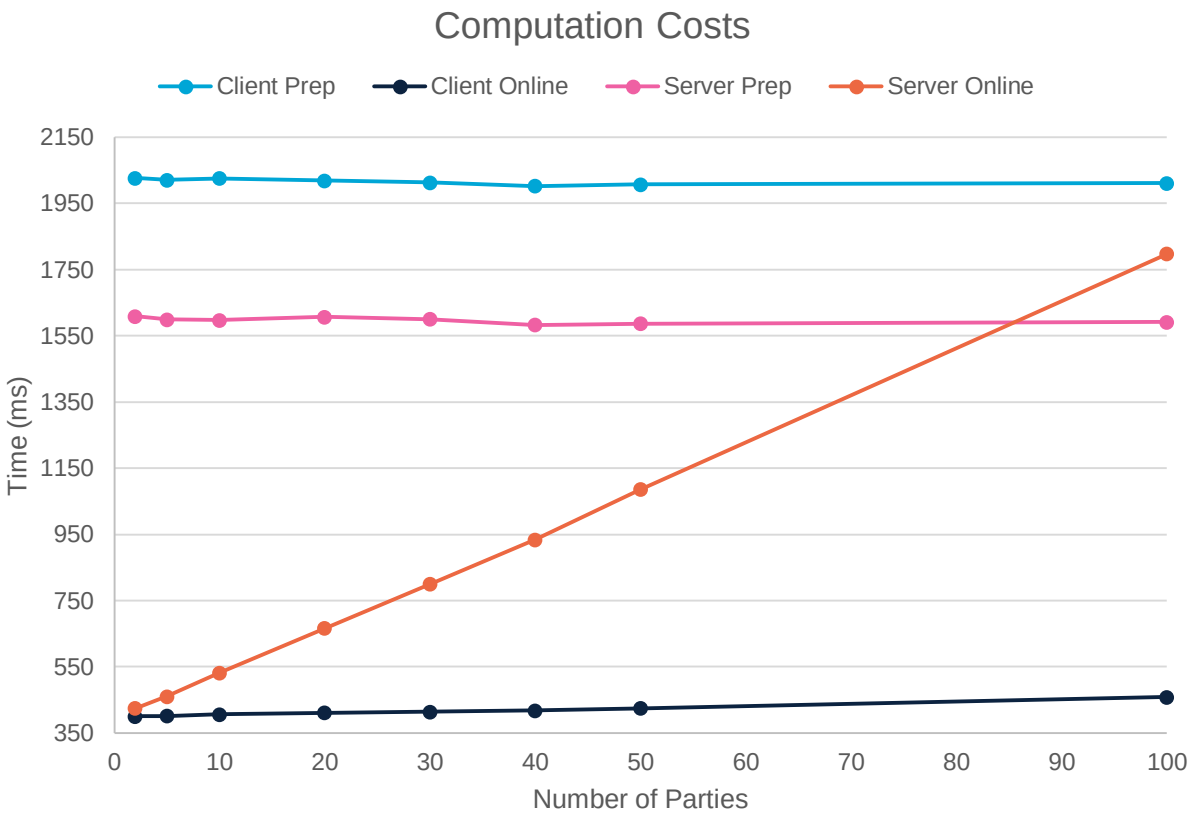
GBFs [10] adaptation to **Ruan et al. [6]** :

- GBF mitigation:
  - The server multiplies the encrypted bins (shares of 1) with a blinded version of its element
  - Adds to the intersection if the decryption is the blinded element

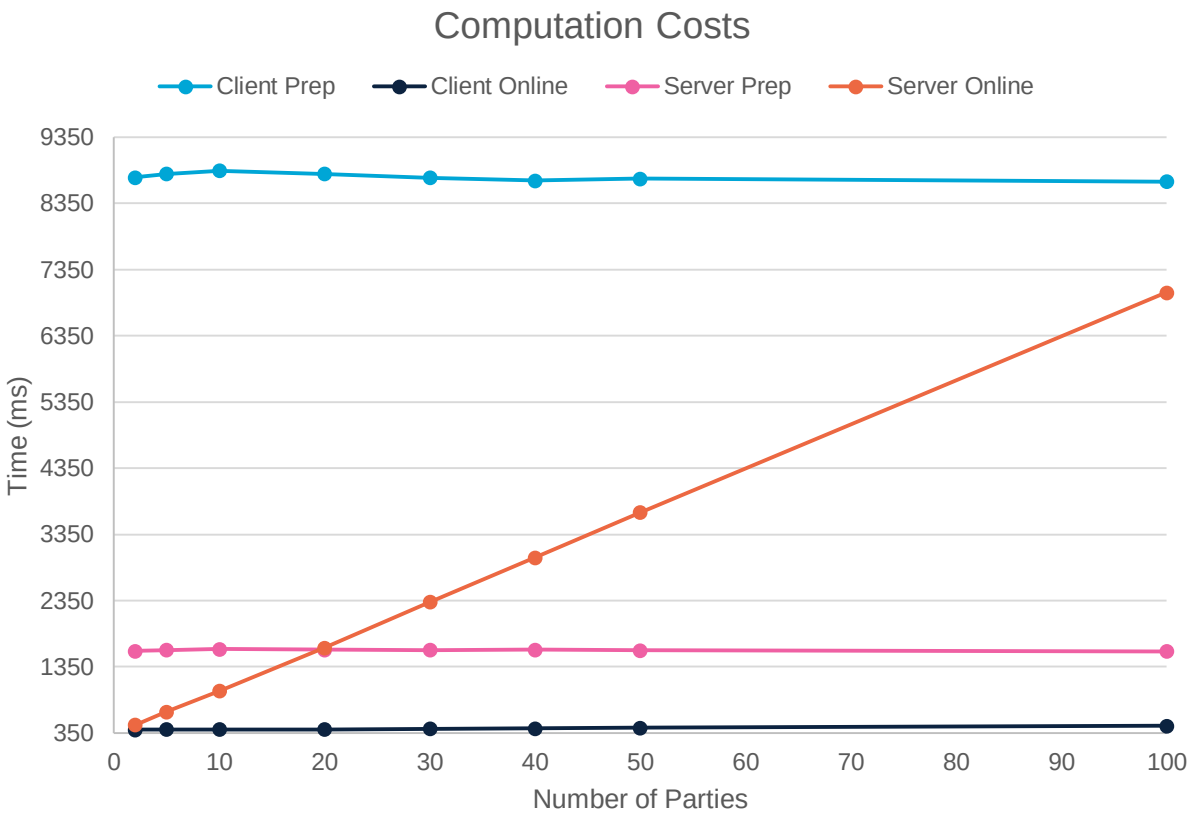
Algorithm 1: $BuildGBF(S, n, m, k, H, \lambda)$

```
input  : A set S, n, m, k, λ, H = {h_0, ...h_{k-1}}
output: An (m, n, k, H, λ)-garbled Bloom filter GBF_S
GBF_S= new m-element array of bit strings;
for i= 0 to m − 1 do
    GBF_S[i]=NULL;                    // NULL is the special symbol that means ''no value''
end
for each x ∈ S do
    emptySlot = −1, finalShare= x;
    for i=0 to k-1 do
        j = h_i(x);                   // get an index by hashing the element
        if GBF_S[j]==NULL then
            if emptySlot ==−1 then
                emptySlot=j;          // reserve this location for finalShare
            else
                GBF_S[j] ←r {0,1}^λ;  // generate a new share
                finalShare=finalShare⊕GBF_S[j];
            end
        else
            finalShare=finalShare⊕GBF_S[j];   // reuse a share
        end
    end
    GBF_S[emptySlot]=finalShare;      // store the last share
end
for i= 0 to m − 1 do
    if GBF_S[i]==NULL then
        GBF_S[i] ←r {0,1}^λ;
    end
end
```

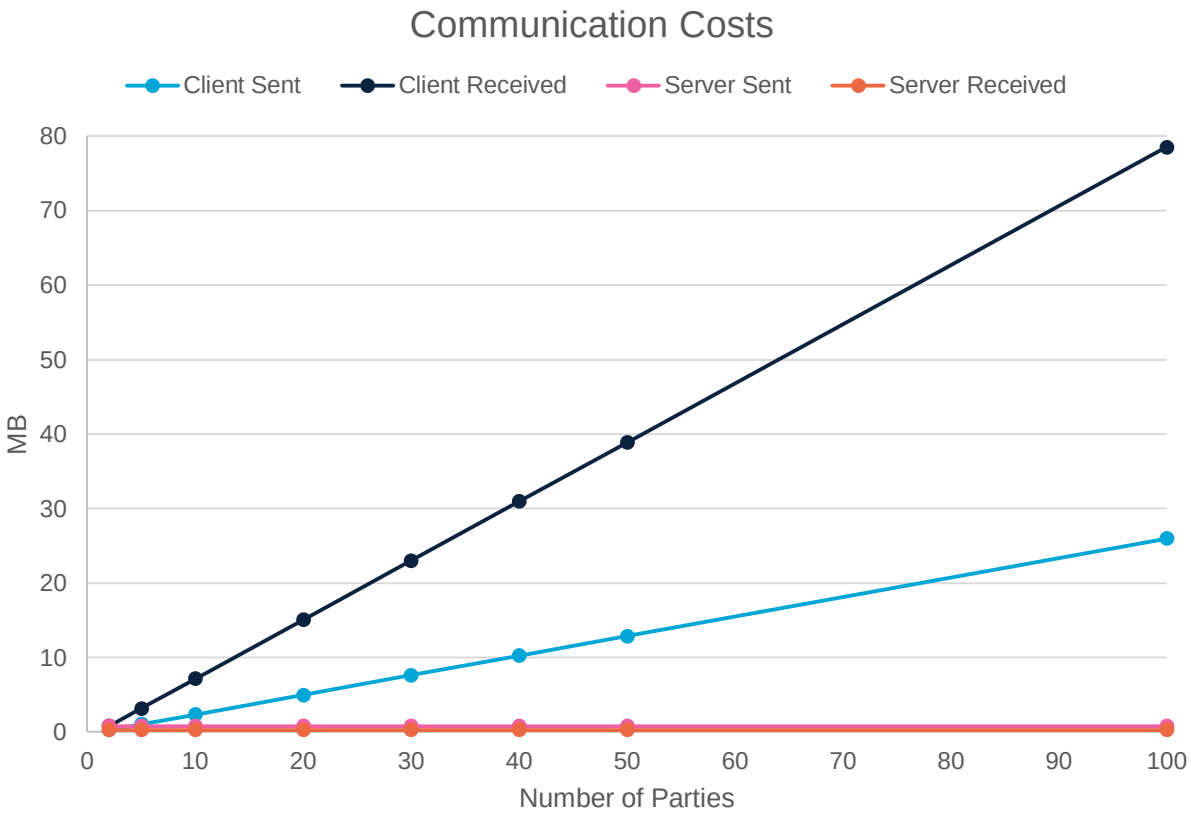
Baseline



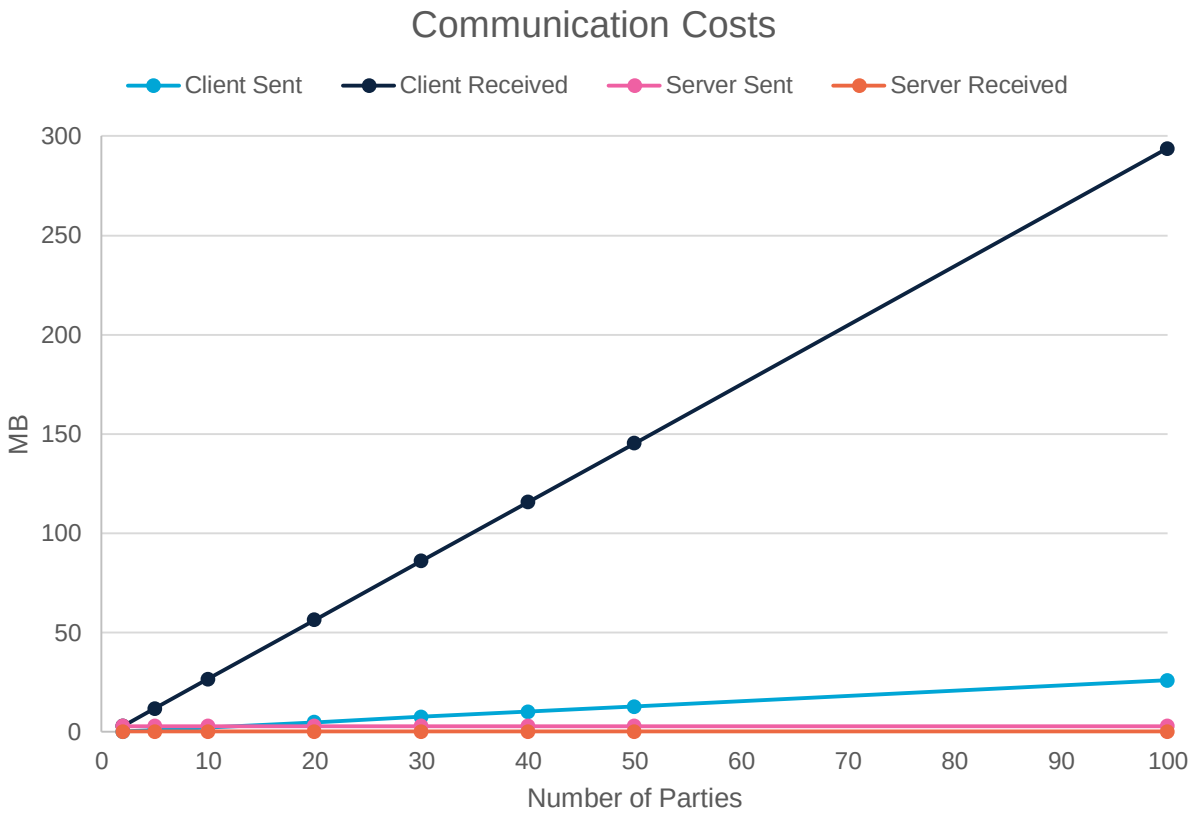
GBF (2^-30)



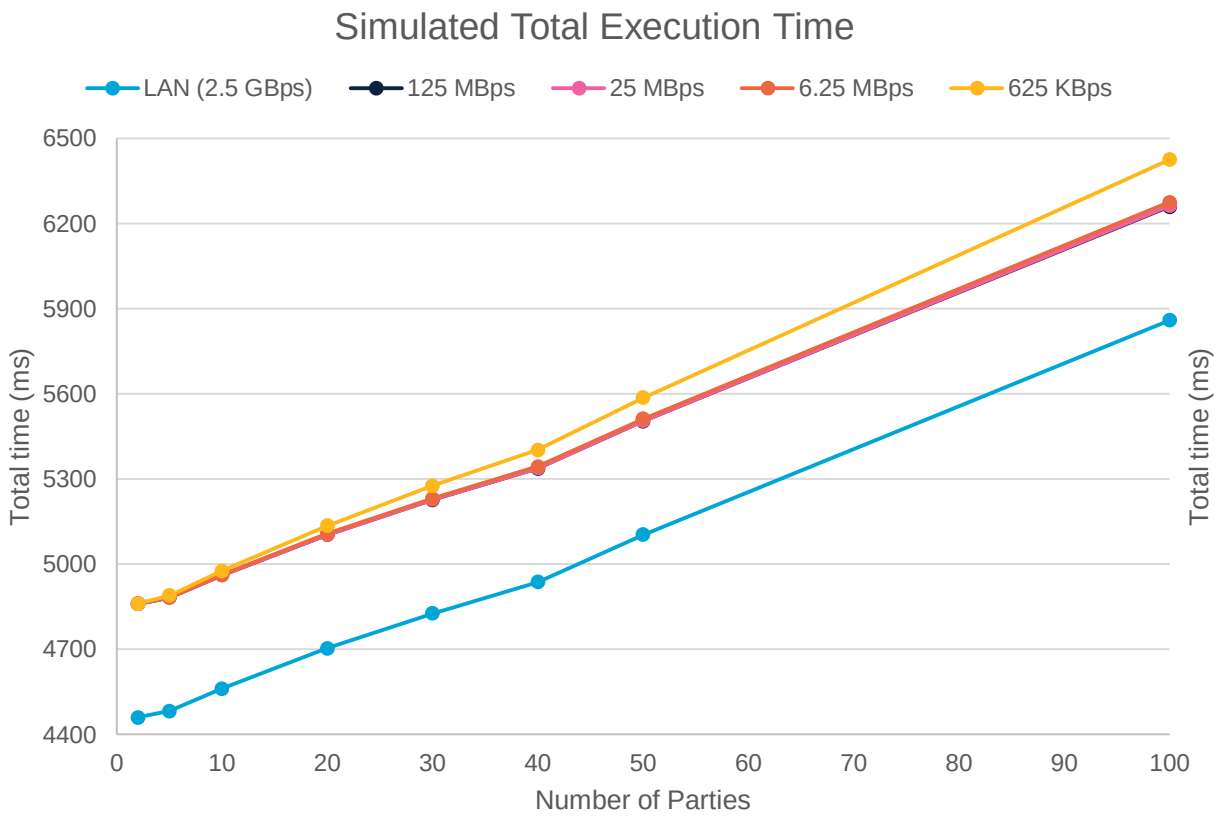
Baseline



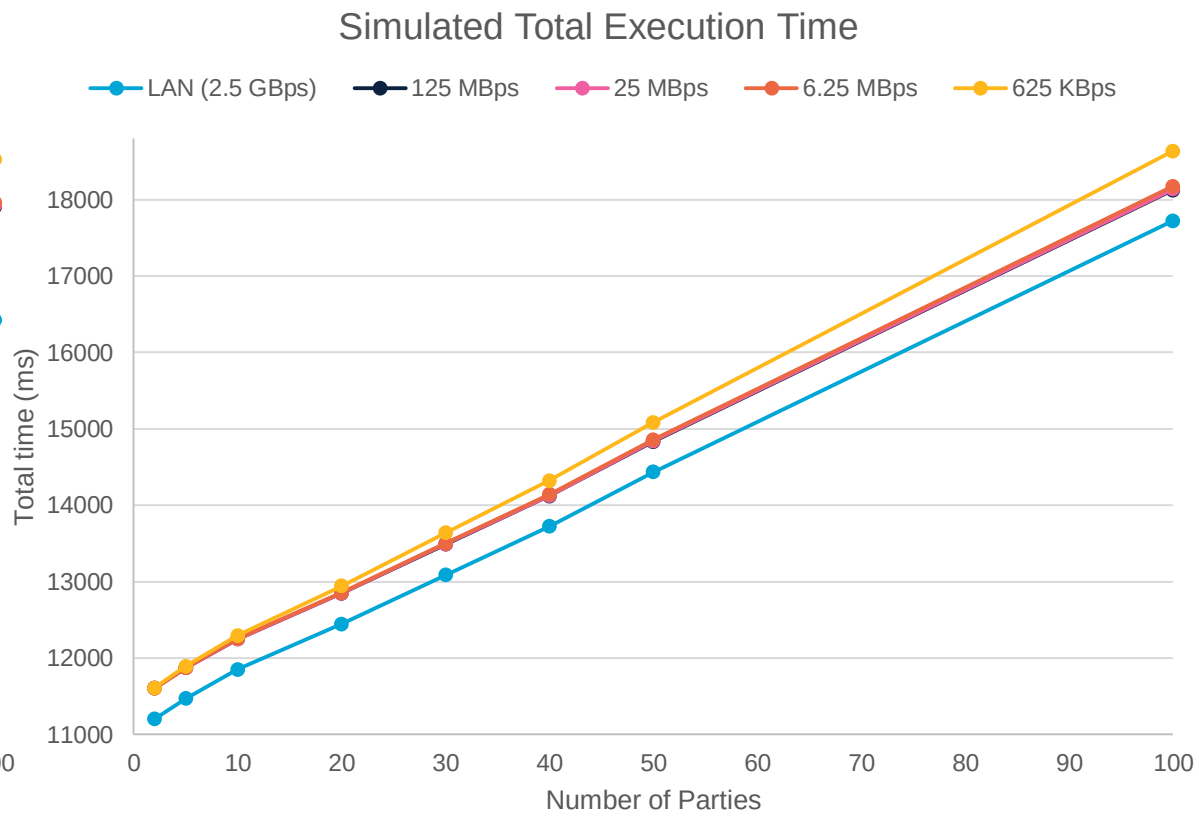
GBF (2^-30)



Baseline



GBF (2^-30)



GBF

Ruan et al. [3] results, **GBF Mitigation (RQ3)**:

(Average) Number of False Negatives for a run of the protocol involving various number of parties (including the server)
Set size clients: 256 elements

Set size server: 1024 elements
FP rate: 2^-7

Parties	Run 1	Run 2	Run 3	Run 4	Run 5	Run 6	Run 7	Run 8	Run 9	Run 10	Mean
2	0	0	0	0	0	0	0	1	0	0	0.1
5	0	0	1	0	0	0	0	0	0	0	0.1
10	0	1	1	0	1	3	2	1	2	0	1.1
20	1	2	3	0	2	0	4	2	1	0	1.5
30	3	1	3	2	3	2	3	5	1	0	2.3
40	3	5	2	3	2	1	2	3	1	4	2.6
50	3	1	4	2	3	3	3	5	5	3	3.2
100	6	10	7	3	8	5	9	8	5	6	6.7

Next Week

- Do some writing on the mitigations
- Continue on the authorized

Thank you!

05-03-2026